

1

THE BIG PICTURE

What is happening inside Draft?

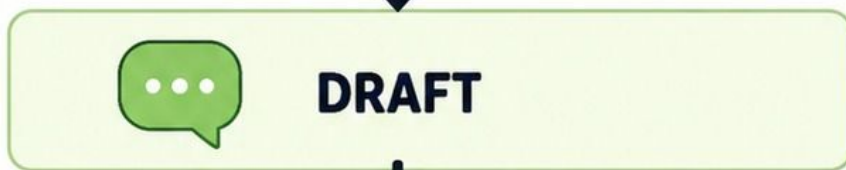


You ask Draft something. Draft's agent thinks, uses tools when needed, gets results, and keeps going until it can give you the best answer.



1

You type a question or give a command to Draft.



2

Draft receives your request and passes it to the agent.



3

The agent (AI) understands what you want and figures out what to do next.



4

The agent repeats this loop as many times as needed to solve the task.

Think → Use Tool →
Get Result → Continue



5

When the agent has everything it needs, it gives you the final answer.



IN SIMPLE WORDS



You ask



Draft receives



Agent works



Uses tools



Gets results



Gives answer

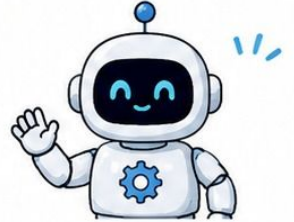
2

WHO IS RUNNING THE AGENT?

Meet AgentRuntime – the engine of Draft



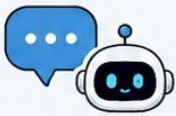
AgentRuntime is the brain and manager of the whole system. It takes your request and drives the agent from start to finish.



AGENTRUNTIME

It is the engine that runs the agent and gets the job done.

1. TALK TO AI



It sends your request to the AI model and receives the AI's response.

2. RUN TOOLS



When the AI wants to use a tool (like read file, run tests, search, etc.), AgentRuntime executes it.

3. TRACK STATE



It keeps track of everything: iterations, tool calls, files changed, test results, current phase and more.

OTHER RESPONSIBILITIES



Manages the conversation



Runs the response → tool → result loop again and again



Handles errors and cancellation



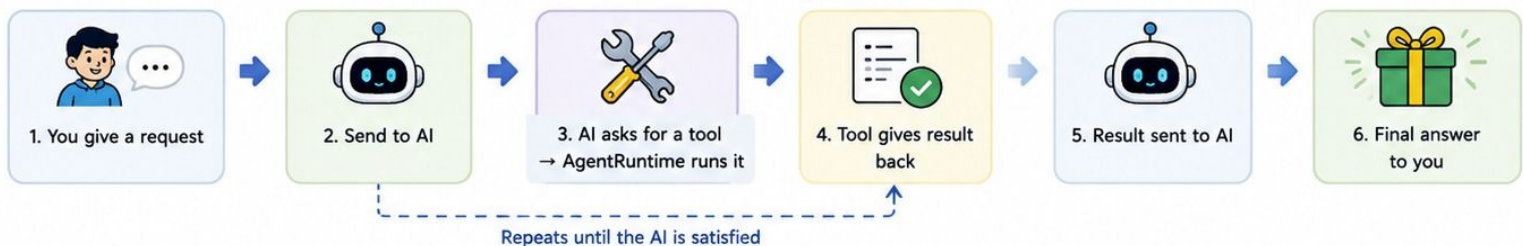
Keeps everything organized and in order

THINK OF AGENTRUNTIME AS...



Same way, AgentRuntime drives the agent to reach the final answer.

HOW AGENTRUNTIME WORKS (THE FLOW)



3 WHAT ARE EVENTS?

Draft tells everyone what is happening!



An event is a message that says,

"Something just happened in the agent!"

Draft creates events for important actions so everyone can know.



COMMON EVENTS IN DRAFT (IN ORDER)

-  **UserMessage**
You send a question or command.
"User asked something."
-  **AgentStarted**
The agent starts working on your request.
"Agent started the task."
-  **AgentPhaseChanged**
The agent moves to a new phase (e.g., Investigation, Execution).
"Agent is now in Investigation."
-  **AgentIterationStarted**
A new think → tool → result iteration begins.
"Starting iteration #2."
-  **ToolStarted**
The agent starts using a tool (like read_file).
"Tool read_file started."
-  **ToolCompleted**
The tool finished successfully and returned a result.
"Tool read_file completed."
-  **FileRead / FileChanged**
Something happened to a file (read, created, changed, deleted).
"Read 120 lines from README.md."
-  **AgentMessage**
The agent sends a text response back (explanation, summary, etc.).
"Here is what I found..."
-  **AgentCompleted**
The agent finished the task successfully.
"Task completed successfully!"

WHY DO WE USE EVENTS?



Everyone can see

The UI, CLI, logs, and tests can all see what's happening.



Detailed tracking

We know every little step the agent takes.



Loose coupling

The agent just creates events; others decide what to do with them.



Better debugging

Events help us understand what went wrong (or right!).

WHAT EVENTS ARE NOT



Not commands

Events don't tell the agent what to do.



Not direct results

Events describe what happened, not the actual data.



Not only for UI

Events are for UI, logs, tests, and future tools too!

EXAMPLE: HOW A SIMPLE TASK LOOKS AS EVENTS

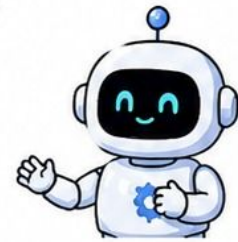


4 WHAT IS AN EVENT BUS?

The delivery system of Draft!



The Event Bus takes events from the agent runtime and delivers them to anyone who wants to listen.
One sender, many receivers!



I carry messages to everyone!

EVENT BUS = POST OFFICE FOR EVENTS

RECEIVERS

(Subscribers)

Anyone can subscribe to receive events.



TUI (Textual UI)
Shows live updates to the user.



CLI
Prints events in the terminal.



Logger
Saves events to log files.



Tests
Check events in automated tests.



Future UI / Tools
Any new system can listen easily!

SENDER

(AgentRuntime)



The agent does work and creates events.

Example Events

- ToolStarted
- ToolCompleted
- FileRead
- AgentMessage
- AgentCompleted
- ...

PUBLISH
(send event)

EVENT BUS

(The Post Office)



Receives events and delivers them to all subscribers.

What it does:

- Keeps a list of subscribers
- Delivers every event
- Stores history (timeline)
- Works safely across threads (emit_threadsafe)

DELIVER

(send to subscribers)

REAL LIFE ANALOGY



You put a letter in the post box (publish). The post office (event bus) delivers it to everyone who is registered to receive it!



You send a letter



Post office receives it



Many people get it



Everyone stays informed!

KEY BENEFITS

- ✓ Decouples sender and receivers
- ✓ Easy to add new listeners
- ✓ Scales to many subscribers
- ✓ Keeps UI responsive
- ✓ Perfect for real-time updates

HOW EVENTS FLOW IN DRAFT



1. AgentRuntime does something



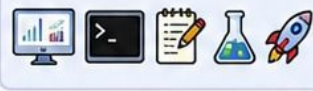
2. Creates an Event (e.g., ToolStarted)



3. Event Bus receives the event



4. Delivers the event to all subscribers



5. Subscribers handle it in their own way



6. You see updates, logs, results, and more!

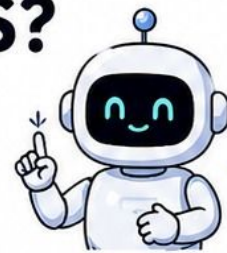
5

WHY DO WE NEED EVENT BUS?

(The Thread + Asyncio Problem)



Draft runs the agent's heavy work in a background thread, but the UI (Textual TUI) runs on the Asyncio event loop. EventBus is the **safe bridge** between them!



Different worlds need a safe way to talk!

THE PROBLEM: TWO DIFFERENT WORLDS

WORLD 1: WORKER THREAD

(Agent doing heavy work)



- ✓ Calls AI model
- ✓ Runs tools (read files, run tests, git, etc.)
- ✓ Takes time
- ✓ Cannot touch UI directly!

Runs in a separate thread (away from the UI)

WORLD 2: ASYNCIO EVENT LOOP

(UI and user interaction)



- ✓ Runs the Textual TUI
- ✓ Handles user input
- ✓ Keeps UI responsive
- ✓ Must never be blocked!

Runs in the main thread (asyncio event loop)

They cannot talk to each other directly!
(Different threads, different rules)

WHAT CAN GO WRONG WITHOUT EVENT BUS?



UI becomes slow
If the agent blocks the asyncio loop.



Race conditions
If both sides try to access data at the same time.



Crashes
If the worker thread touches UI objects directly.



Bad user experience
Laggy, freezing, or unresponsive UI.

THE SOLUTION: EVENT BUS IS THE SAFE BRIDGE

1. WORKER THREAD

(AgentRuntime)



Does work and creates events

2. Emit Event from thread

Thread-safe call
`emit_threadsafe(event)`

3. EVENT BUS

(The Bridge)



Receives events safely and puts them into the asyncio world.

4. Deliver Event safely to loop

`call_soon_threadsafe()`
→ schedule in event loop

5. ASYNCIO EVENT LOOP

(TUI / Main Thread)



Gets the event and updates the UI

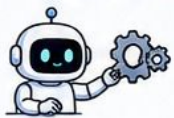
RESULT



Agent works hard in the background.
UI stays **smooth and responsive**.
Everyone is happy!

HOW `emit_threadsafe()` WORKS (BEHIND THE SCENES)

1 Worker thread calls: `emit_threadsafe(event)`



2 EventBus uses the asyncio loop:

```
loop.call_soon_threadsafe(
    loop.create_task,
    _deliver(event)
)
```

3 This schedules `_deliver(event)` to run inside the event loop.



4 `_deliver(event)` puts the event into all queues and notifies subscribers.



5 TUI reads the event from its queue and updates the screen!



EXAMPLE FLOW (ToolStarted Event)

AgentRuntime (Worker Thread)

`ToolStarted(tool_name="read_file")`



`event_bus.emit_threadsafe(event)`

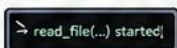
EventBus (Bridge)

`call_soon_threadsafe()`
→ `_deliver(event)`



Asyncio Loop (TUI)

Queue gets event
TUI displays:
"→ `read_file(...)` started"



KEY TAKEAWAY



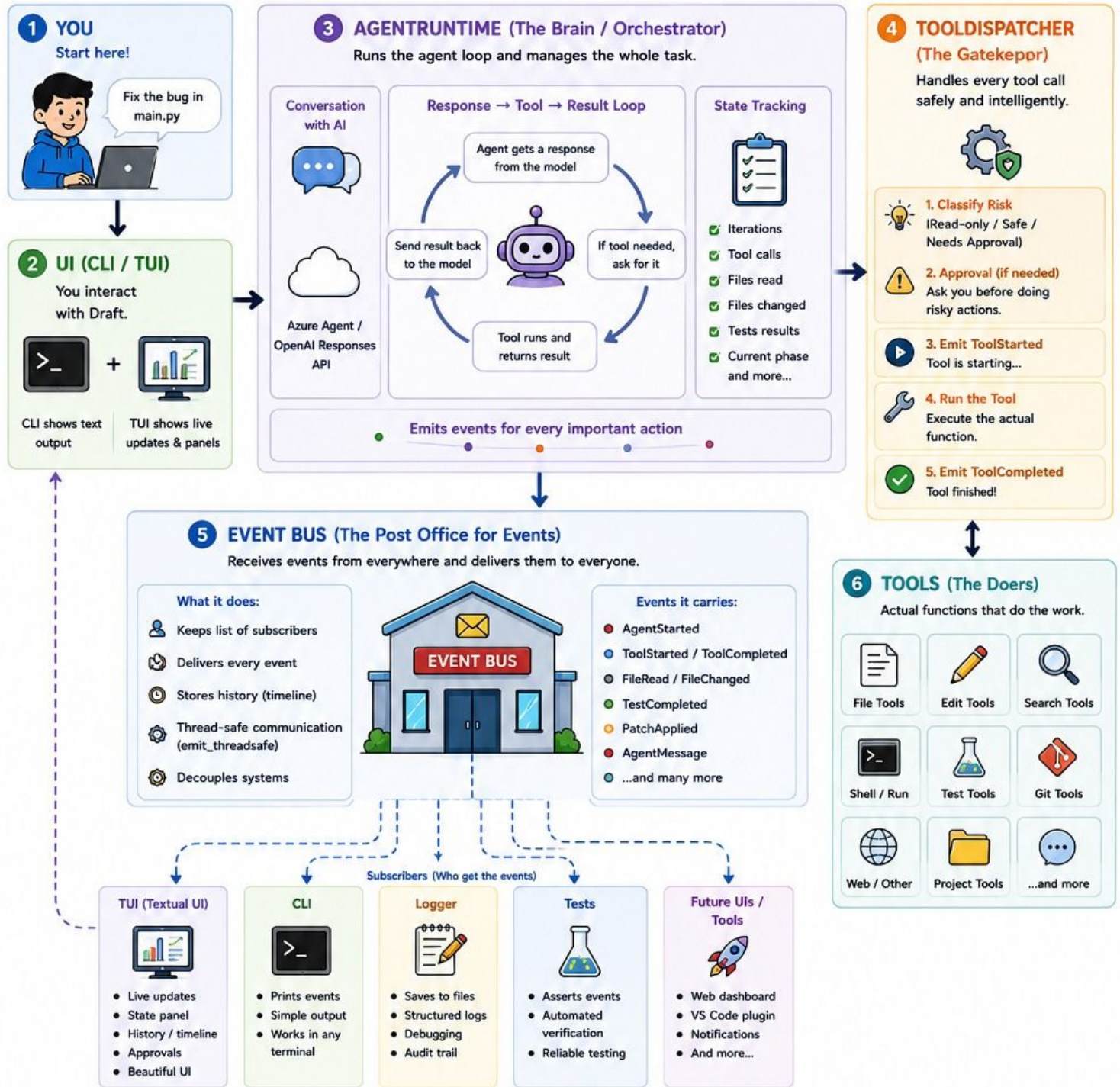
- ✓ Threads can't touch the UI.
- ✓ Asyncio loop can't be blocked.
- ✓ EventBus is the safe bridge.
- ✓ `emit_threadsafe()` makes cross-world communication safe.
- ✓ That's why Draft's UI stays smooth while the agent works!

6 THE COMPLETE DRAFT ARCHITECTURE

How everything works together!



Draft is an AI coding assistant. Many parts work together so the agent can think, act, and keep you informed in real time.



THE END-TO-END FLOW (Putting it all together)



Events are emitted at every step and delivered to all subscribers!

WHY THIS ARCHITECTURE IS POWERFUL

- ✓ Clean separation of concerns
- ✓ UI stays responsive
- ✓ Easy to add new UIs or tools
- ✓ Perfect for real-time updates, logging & testing
- ✓ Safe, scalable, and future-ready



IN ONE SENTENCE

AgentRuntime thinks and coordinates, ToolDispatcher acts safely, EventBus communicates, and UIs & other tools stay informed — so you always know what Draft is doing!



KEY ROLES (Quick Recap)

- AgentRuntime → Orchestrates & thinks
- ToolDispatcher → Controls tool execution
- EventBus → Distributes events
- Tools → Do the real work
- UI / CLI / Others → Show, log, test, and more